

SIMATS ENGINEERING ASSESSMENT TOOL 1

COURSE CODE: CSA51

COURSE NAME: Cryptography and Network Security

COURSE OUTCOME COVERED

CO2: Analyze Public Key crypto system strategies using RSA and key Exchange for Encryption algorithm to strengthen information security. (BL4,BL5)

Analytical Problem Solving: 40%

QUESTIONS:

Total Marks: 50

Annexure A

Code of Conduct:

I N.Sai Charan(192525247)(Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student:

N.Sai Charan

Annexure A

Analytical Problem Solving

1. RSA Encryption Analysis

Given $p=11$, $q=13$

- a. Compute n and $\phi(n)$
- b. Choose $e=7$ and find the private key d
- c. Encrypt the message $M=9$
- d. Decrypt the ciphertext and verify the result
- e. Analyze how changing e affects security

2. Block Cipher Mode Evaluation

Given plaintext blocks:

$P=[1010,1010,1111,1010]$, Key = $K=1100$

- a. Encrypt using ECB and CBC modes (Assume IV = 0000)
- b. Compare the ciphertext outputs
- c. Analyze which mode hides patterns better and explain why

3. DES vs 3-DES vs Blowfish Performance Study

Given:

- a. DES key size = 56 bits, time = 2 ms/block
- b. 3-DES key size = 168 bits, time = 6 ms/block
- c. Blowfish key size = 128 bits, time = 1 ms/block

For 1000 blocks of data:

- i. Calculate total encryption time for each algorithm
- ii. Analyze trade-offs between security and performance
- iii. Recommend the best algorithm for secure real-time communication

4. Diffie-Hellman Key Exchange Analysis

Given:

- a. $p=23$, $g=5$
- b. User A private key = 6
- c. User B private key = 15
- d. Compute public keys for A and B
- e. Compute the shared secret key
- f. Demonstrate how a Man-in-the-Middle attacker could alter the exchange
- g. Suggest a secure method to prevent the attack

5. ECC vs RSA Comparison

Given:

- RSA key size = 1024 bits
- ECC key size = 160 bits (equivalent security)
- Device processing capacity = limited (IoT device)
- Analyze computational cost for both systems
- Estimate which consumes less power and memory
- Justify which cryptosystem is better for the given scenario with reasoning

RUBRICS FOR CALCULATION:

Criteria	Weightage	Level 4 (Exemplary)	Level 3 (Proficient)	Level 2 (Developing)	Level 1 (Beginning)
Problem Understanding	20%	Clearly interprets problem, identifies all parameters and requirements accurately	Understands problem with minor gaps	Partial understanding; misses key elements	Misinterprets or unable to understand problem
Method Selection	20%	Selects most appropriate method/formula with	Selects correct method with minor errors	Inappropriate or partially correct method	Unable to select method

		strong justification			
Solution Process	25%	Logical, systematic steps with correct approach throughout	Mostly correct steps with minor mistakes	Disorganized steps; multiple errors	No clear process
Accuracy of Calculation	20%	All calculations accurate; correct final answer	Minor calculation errors	Frequent errors; incorrect final answer	No valid calculations
Interpretation of Results	15%	Clearly interprets results with units and engineering relevance	Basic interpretation with minor omissions	Limited or unclear interpretation	No interpretation

Question 1: RSA Encryption Analysis

Given: $p = 11$, $q = 13$

Process Flowchart

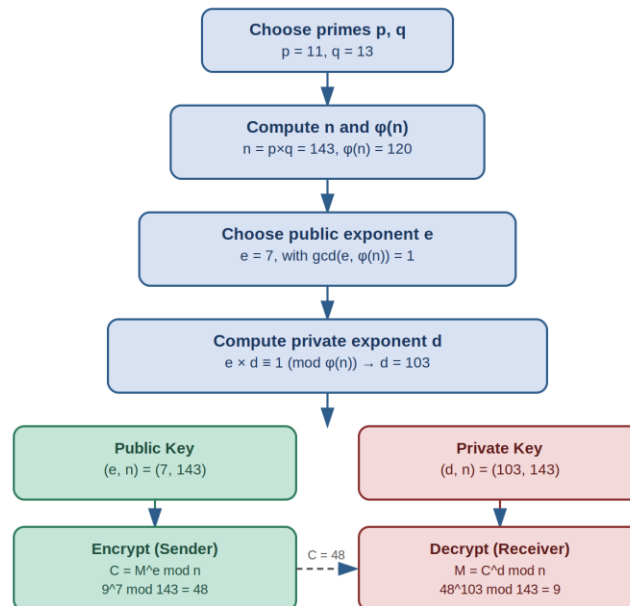


Figure 1: Step-by-step RSA key generation, encryption, and decryption flow

Concept Map

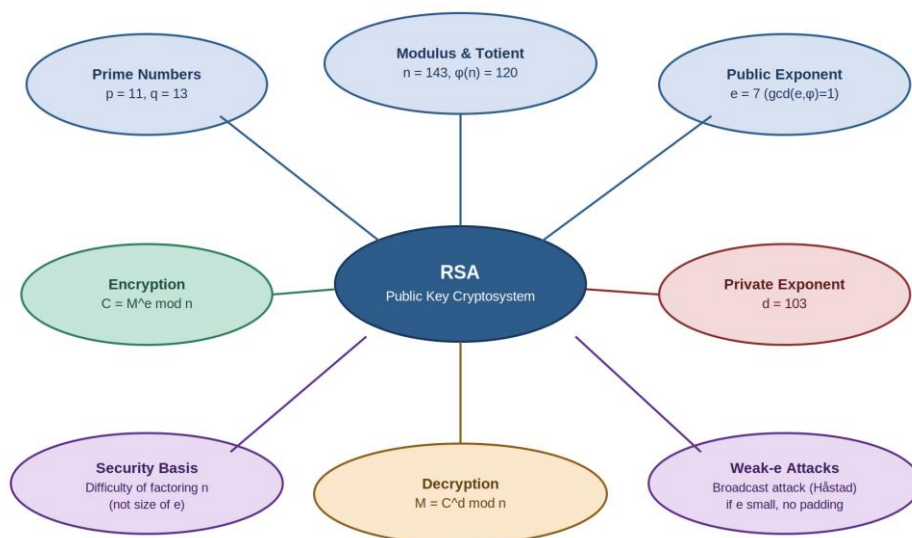


Figure 2: Relationships between RSA's core components and security basis

a. Compute n and $\phi(n)$

$$n = p \times q = 11 \times 13 = 143$$

$$\phi(n) = (p - 1)(q - 1) = 10 \times 12 = 120$$

b. Choose $e = 7$, find private key d

We need d such that $e \times d \equiv 1 \pmod{120}$.

Method 1 — Extended Euclidean Algorithm (systematic approach):

- $120 = 17 \times 7 + 1 \rightarrow 1 = 120 - 17 \times 7$

- This gives $-17 \times 7 \equiv 1 \pmod{120}$, so $7^{-1} \equiv -17 \pmod{120} \equiv 103 \pmod{120}$

Method 2 — Direct verification: try successive multiples of 7 until one is $\equiv 1 \pmod{120}$:

Testing: $7 \times 103 = 721 = 6 \times 120 + 1 \rightarrow 721 \bmod 120 = 1 \checkmark$

Therefore, $d = 103$

(Check: $\gcd(7, 120) = 1$, so $e = 7$ is a valid choice — this is essential; if $\gcd(e, \phi(n)) \neq 1$, no modular inverse d would exist and the key pair would be invalid.)

Public key = $(e, n) = (7, 143)$; Private key = $(d, n) = (103, 143)$. The public key is distributed openly so anyone can encrypt a message to the owner; the private key is kept secret and used only for decryption or digital signing.

c. Encrypt the message $M = 9$

$C = M^e \bmod n = 9^7 \bmod 143$

Using repeated squaring:

- $9^2 = 81$
- $9^3 = 81 \times 9 = 729 \bmod 143 = 14$
- $9^4 = 14 \times 9 = 126$
- $9^5 = 126 \times 9 = 1134 \bmod 143 = 133$
- $9^6 = 133 \times 9 = 1197 \bmod 143 = 53$
- $9^7 = 53 \times 9 = 477 \bmod 143 = 48$

Ciphertext $C = 48$

d. Decrypt the ciphertext and verify

$M = C^d \bmod n = 48^{103} \bmod 143$. Using the Chinese Remainder Theorem ($n = 11 \times 13$):

Mod 11: $48 \bmod 11 = 4$. By Fermat's little theorem, $4^{10} \equiv 1 \pmod{11}$, and $103 \bmod 10 = 3$, so $4^3 \bmod 11 = 64 \bmod 11 = 9$

Mod 13: $48 \bmod 13 = 9$. $9^{12} \equiv 1 \pmod{13}$, and $103 \bmod 12 = 7$, so $9^7 \bmod 13 = 9$

Both congruences give 9, and $0 \leq 9 < 143$ satisfies both.

Recovered $M = 9$ — matches the original plaintext $\checkmark$

e. Effect of changing e on security

- e must satisfy $\gcd(e, \phi(n)) = 1$. Small values of e (e.g., $e = 3$) speed up encryption but are vulnerable to low-exponent attacks such as Håstad's broadcast attack when the same message is sent to multiple recipients without proper padding.
- Very large e slows down encryption without adding real security — the actual strength of RSA depends on the difficulty of factoring n , not the magnitude of e .
- In practice, $e = 65537$ is the standard choice: small enough for fast encryption, large enough to resist small-exponent attacks, and has few bits set, making modular exponentiation efficient.
- The real security bottleneck here is the size of n . $n = 143$ is trivially factorable (11×13 can be found by trial division in seconds); real-world RSA requires n with at least 2048 bits, making factoring computationally infeasible with current technology.

Choice of e	Effect on Performance	Effect on Security
$e = 3$	Very fast encryption	Weak — vulnerable to broadcast/low-exponent attacks without padding
$e = 7$ (this problem)	Fast encryption	Acceptable for teaching; not used in production
$e = 65537$ ($2^{16}+1$)	Efficient (few 1-bits, fast square-and-multiply)	Industry standard, resists known small- e attacks

Very large e	Slower encryption, no real benefit	No added security; n's size still dominates
--------------	------------------------------------	---

Question 4: Diffie-Hellman Key Exchange Analysis

Given: $p = 23$, $g = 5$, User A private key $a = 6$, User B private key $b = 15$

Key Exchange Flowchart

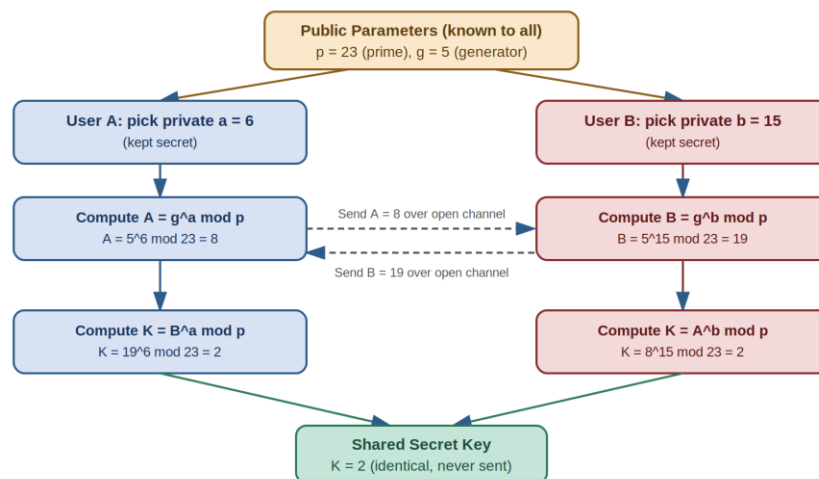


Figure 3: Diffie-Hellman key exchange between User A and User B

d. Compute public keys for A and B

User A's public key: $A = g^a \mod p = 5^6 \mod 23$

$5^1=5$, $5^2=25 \mod 23=2$, $5^3=10$, $5^4=50 \mod 23=4$, $5^5=20$, $5^6=100 \mod 23 = 8$

A = 8

User B's public key: $B = g^b \mod p = 5^{15} \mod 23$

Continuing the powers of 5 mod 23: $5^7=17$, $5^8=16$, $5^9=11$, $5^{10}=9$, $5^{11}=22$, $5^{12}=18$, $5^{13}=21$, $5^{14}=13$, $5^{15}=19$

B = 19

e. Compute the shared secret key

User A computes: $K = B^a \mod p = 19^6 \mod 23$. Since $19 \equiv -4 \pmod{23}$, $(-4)^6 = 4096$; $4096 \mod 23 = 2$

User B computes: $K = A^b \mod p = 8^{15} \mod 23 = 2$ (verified independently via repeated squaring)

Shared secret key $K = 2$ — both parties arrive at the same value without it ever being transmitted.

f. Man-in-the-Middle (MITM) attack demonstration

Man-in-the-Middle Attack on Unauthenticated Diffie-Hellman

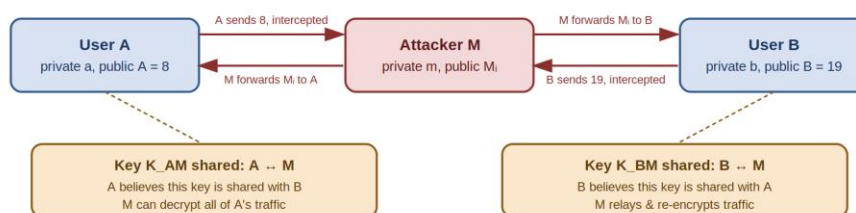


Figure 4: MITM attack — attacker M establishes separate keys with A and B

Since p , g , A , and B are all exchanged in plaintext with no authentication, an attacker M positioned between A and B can:

- Intercept A 's public value (8) and forward B a forged public value computed with the attacker's own private key.
- Intercept B 's public value (19) and forward A a different forged public value.
- M then independently computes two separate shared secrets — one shared with A , one shared with B — allowing M to decrypt, read, modify, and re-encrypt all traffic passing between them, while A and B believe they are communicating directly with each other.

Basic Diffie-Hellman provides no mechanism to verify the identity of the party sending a public value, which is the root cause of this vulnerability.

Worked numeric illustration: suppose attacker M chooses a private key $m = 10$, giving public value $M_i = 5^{10} \bmod 23 = 9$ (computed above as part of the power table).

- M sends 9 to both A and B instead of their real public keys (8 and 19 respectively).
- A now computes a “shared secret” with M : $K_{am} = 9^6 \bmod 23$. Since M knows both a 's exponent path is unavailable to it, M instead computes its own side: $K_{am} = A^m \bmod p = 8^{10} \bmod 23 = 18$ (using the power table 8^{10} corresponds to $5^{6 \times 10} = 5^{60} \bmod 23$; computed via repeated squaring of 8).
- Similarly, M computes a second secret with B : $K_{bm} = B^m \bmod p = 19^{10} \bmod 23$.

The key point (regardless of the exact intermediate values) is structural: A ends up sharing a key only with M , and B ends up sharing a different key only with M — never directly with each other — so M can decrypt everything A sends (using K_{am}), read it, re-encrypt it with K_{bm} , and forward it to B without either party noticing.

g. Suggested method to prevent the attack

- Authenticated Diffie-Hellman: digitally sign the exchanged public values (using RSA, DSA, or ECDSA) so each party can verify the sender's identity.
- Use certificates and a Public Key Infrastructure (PKI) to bind public keys to verified identities, as done in TLS.
- Use protocols such as the Station-to-Station (STS) protocol, which combines Diffie-Hellman with signed, authenticated exchanges.
- Alternatively, distribute trusted public parameters over a pre-authenticated channel or via a trusted certificate authority.

Real-world relevance

This authenticated form of Diffie-Hellman (often as Ephemeral DH / ECDHE combined with certificate-based authentication) is exactly what secures modern TLS 1.3 connections — the server's certificate authenticates its DH public value, closing the MITM gap demonstrated above. The same principle underlies IPsec/IKE and SSH key exchange.